

day 18

day 4 node

Obsidian Notes: Node.js Day 4 Demo (Mongoose, Multer, ImageKit & Validation)

💡 المفهوم الأساسي للديمو يمثل هذا الديمو النقلة من التخزين المحلي في ملفات JSON إلى قاعدة بيانات MongoDB عبر مكتبة Mongoose، مع إضافة قدرات رفع الملفات وسحابياً (Multer + ImageKit)، والتحقق من صحة المدخلات (Joi Validation)، وإدارة أخطاء قواعد البيانات المركزية.

1. نقطة الانطلاق وإعداد البيئة (index.js & package.json)

تأتي نقطة البداية لتشغيل السيرفر وربطه بقاعدة البيانات وال Middlewares الأساسية:

JavaScript

```
require("dotenv").config(); // 🔑 تحميل المتغيرات البيئية من ملف .env
const express = require("express");
const app = express();
const userRouter = require("./routes/user");
const errorHandler = require("./middleware/error");
const notFoundHandler = require("./middleware/not-found");
const mongoose = require("mongoose");
const morgan = require("morgan");
const cors = require("cors");

// 🛠 1. Middlewares العامة
app.use(express.json()); // Parsing لـ JSON Body
app.use("/uploads", express.static("uploads")); // تقديم الملفات المرفوعة محلياً
app.use(morgan("dev")); // Logging لـ HTTP Requests
app.use(cors()); // السماح بالاتصال من واجهات مختلفة (Cross-Origin)

// 🚦 2. Routing & Handlers
app.use(userRouter);
app.use(notFoundHandler);
app.use(errorHandler);

// 🌐 3. Server Setup & Database Connection
app.listen(process.env.PORT || 3000, () => {
  console.log(`Server is running on http://localhost:${process.env.PORT}`);
  mongoose
```

```
.connect("mongodb://127.0.0.1:27017/node-js-iti")
.then(() => console.log("DB Connected!"));
});
```

🔗 أهم المكتبات الجديدة في package.json

- **mongoose** : Schemas وإنشاء الـ MongoDB للتعامل مع.
- **multer** : multipart/form-data لاستقبال الملفات والصور المبعوثة كـ.
- **imagekit** : فقط URLs للرفع السحابي للصور وتخزين الـ.
- **joi** : (Request Body Validation) للتحقق من صحة المدخلات.

2. رفع الملفات السحابي والمحلي (upload-image.js & image-kit.js)

تتم عملية الرفع على مرحلتين أساسيتين:

Plaintext

```
[ Client ] —(FormData)—> [ Multer (Memory Storage) ] —(Buffer)—> [
ImageKit Middleware ] —(URL)—> [ Controller ]
```

أ) استقبال الملف بـ (Multer (middleware/upload-image.js)

:

- **diskStorage** : uploads . حفظ الصور محلياً داخل مجلد
- **memoryStorage** : مؤقت تمهيداً لرفعها سحابياً Buffer كـ RAM حفظ الصور في الـ.

JavaScript

```
const multer = require("multer");
const path = require("path");

const diskStorage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "uploads"),
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, `${Date.now()}-${file.fieldname}${ext}`);
  },
});

const memoryStorage = multer.memoryStorage();
```

```

module.exports = {
  uploadOnDisk: multer({ storage: diskStorage }),
  uploadOnMemory: multer({ storage: memoryStorage }),
};
```[cite: 18]

```

#### \*\*رفع الصورة على ImageKit (`middleware/image-kit.js`)[cite: 20]:\*\*  
 ثم يضع رابط الصورة، ImageKit الخاص بالصورة ويقوم برعه على خدمة Buffer يأخذ الـ  
 Controller[cite: 20] ليتمرر للـ `req.images` الناتج في

```

```javascript
const ImageKit = require("imagekit");

const imageKit = new ImageKit({
  publicKey: process.env.IMAGEKIT_PUBLIC_KEY,
  privateKey: process.env.IMAGEKIT_PRIVATE_KEY,
  urlEndpoint: process.env.IMAGEKIT_URL_ENDPOINT,
});

const uploadImageKit = (isMultiple = false, folder = "uploads") => {
  return async (req, res, next) => {
    if ((!isMultiple && !req.file) || (isMultiple && req.files.length === 0))
    {
      return next();
    }
    const files = isMultiple ? req.files : [req.file];

    const uploadPromises = files.map((file) => {
      return imageKit.upload({
        file: file.buffer,
        fileName: `${Date.now()}-${file.fieldname}`,
        folder: folder,
      });
    });

    const images = await Promise.all(uploadPromises);
    req.images = images.map((image) => image.url); // فقط URLs حفظ الـ
    next();
  };
};

module.exports = uploadImageKit;
```[cite: 20]

```

---

### ### ● 3. والعلاقات Schema تصميم الـ (`model/post.js`)

بالـ Post بربط كل **\*\*One-to-Many Relationship\*\*** كيفية تطبيق `Post.js` يوضح ملف `ObjectId` الخاصية `User` الـ collection الخاص بالمؤلف في `ref` [cite: 15]:

```
```javascript
const mongoose = require("mongoose");

const postSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  image: { type: String },
  author: {
    type: mongoose.Schema.Types.ObjectId, // 🔗 ربط باستخدام ObjectId
    ref: "User", // 🏠 إشارة لـ User Model
    required: true,
  },
});

const Post = mongoose.model("Post", postSchema);
module.exports = Post;
``` [cite: 15]
```

### ### ● 4. تسلسل المسارات والميدل ويرز (`routes/user.js`)

بالترتيب الدقيق أثناء إضافة Middlewares تتجلى قوة المعمارية في ترتيب تنفيذ الـ `POST /users` [cite: 14]:

```
```javascript
router.post(
  "/users",
  uploadOnMemory.single("img"), // 1 Multer في الـ الصورة في الـ
  Memory [cite: 14, 18]
  uplaodImageKit(false, "user-iti"), // 2 ImageKit الـ ويضع الـ URL
  req.images [cite: 14, 20]
  validate(createUserSchema), // 3 Joi في البيانات في
  req.body [cite: 14]
  createUser // 4 MongoDB يحفظ المستند في Controller
);
``` [cite: 14]
```

### ● 5. Central Error Handling معالجة أخطاء المونجو وال  
(`middleware/error.js`)

Status الشائعة ويرجع الـ Mongoose ليتعرف على أخطاء `errorHandler` تم التوسع في  
[cite: 19]: المناسب بدلاً من انهيار السيرفر Code

```
```javascript
const AppError = require("../utils/AppError");

const errorHandler = (err, req, res, next) => {
  console.error(err);

  // 1 أخطاء محددة مسبقاً عبر [cite: 19]
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({ message: err.message });
  }

  // 2 MongoDB أخطاء القيم المكررة في (مثل تكرار الإيميل - Code 11000)[cite: 19]
  if (err.code === 11000) {
    const field = Object.keys(err.keyPattern)[0];
    return res.status(400).json({ message: `Duplicate value for ${field}` });
  }

  // 3 Validation أخطاء الـ Mongoose Schema الخاصة بـ Validation أخطاء الـ [cite: 19]
  if (err.name === "ValidationError") {
    return res.status(400).json({ message: err.message });
  }

  // 4 Invalid ID Format (CastError) أخطاء الـ [cite: 19]
  if (err.name === "CastError") {
    return res.status(400).json({ message: "invalid id format" });
  }

  // 5 أي خطأ غير متوقع [cite: 19]
  res.status(500).json({ message: "Internal server error" });
};

module.exports = errorHandler;
```[cite: 19]

```

### 📄 في الديمو Request ملخص تسلسل خط سير الـ

| الخطوة | المكون المسئول | الوظيفة |

| :--- | :--- | :--- |  
| \*\*1\*\* | `express.json()` / `Multer` | تفكيك البيانات المبعوثة من العميل سواء  
JSON أو Multipart[cite: 11, 18]. |  
| \*\*2\*\* | `uploadImageKit` | رفع الصور سحابياً وإلحاق رابط الصورة بـ  
`req.images`[cite: 20]. |  
| \*\*3\*\* | `joi-validate` | Database[cite: 14]. |  
| \*\*4\*\* | `Controller` | التأكيد من صحة المدخلات قبل الوصول للـ  
(`User.create` / `Post.create`) التعامل مع الموديل |  
| | | لإتمام العملية |  
| \*\*5\*\* | `errorHandler` | الالتيقظ التلقائي لأي خطأ (Duplicate Email,  
CastError, etc.) وإرجاع 400 أو